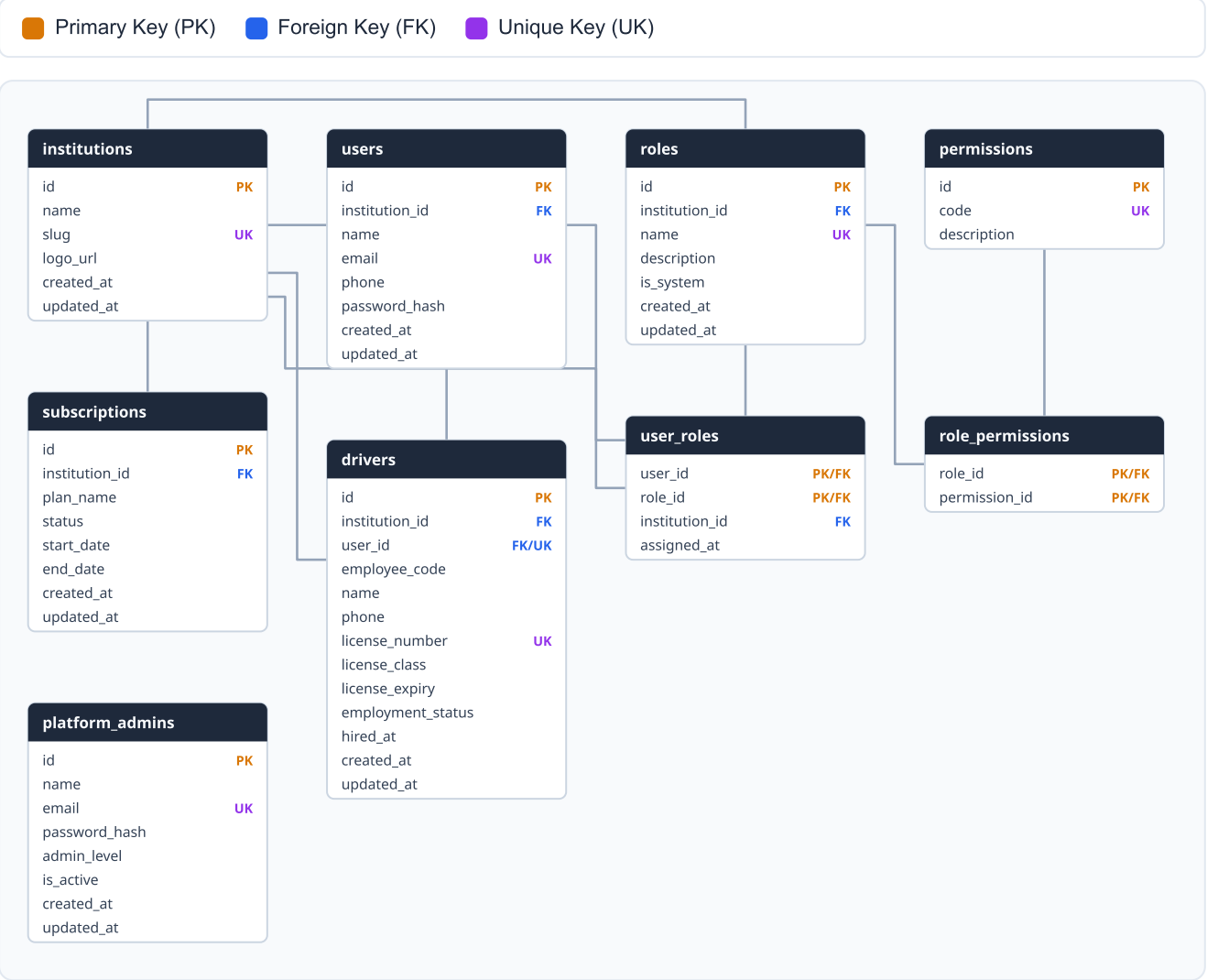
FleetSync Database Schema Map

Version 2.0 • Multi-Tenant SaaS Architecture

This document presents the visual Entity-Relationship (ER) models for the FleetSync platform. The architecture enforces strict tenant isolation through composite keys and shared `institution_id` constraints, grouping logical domains into four primary views.

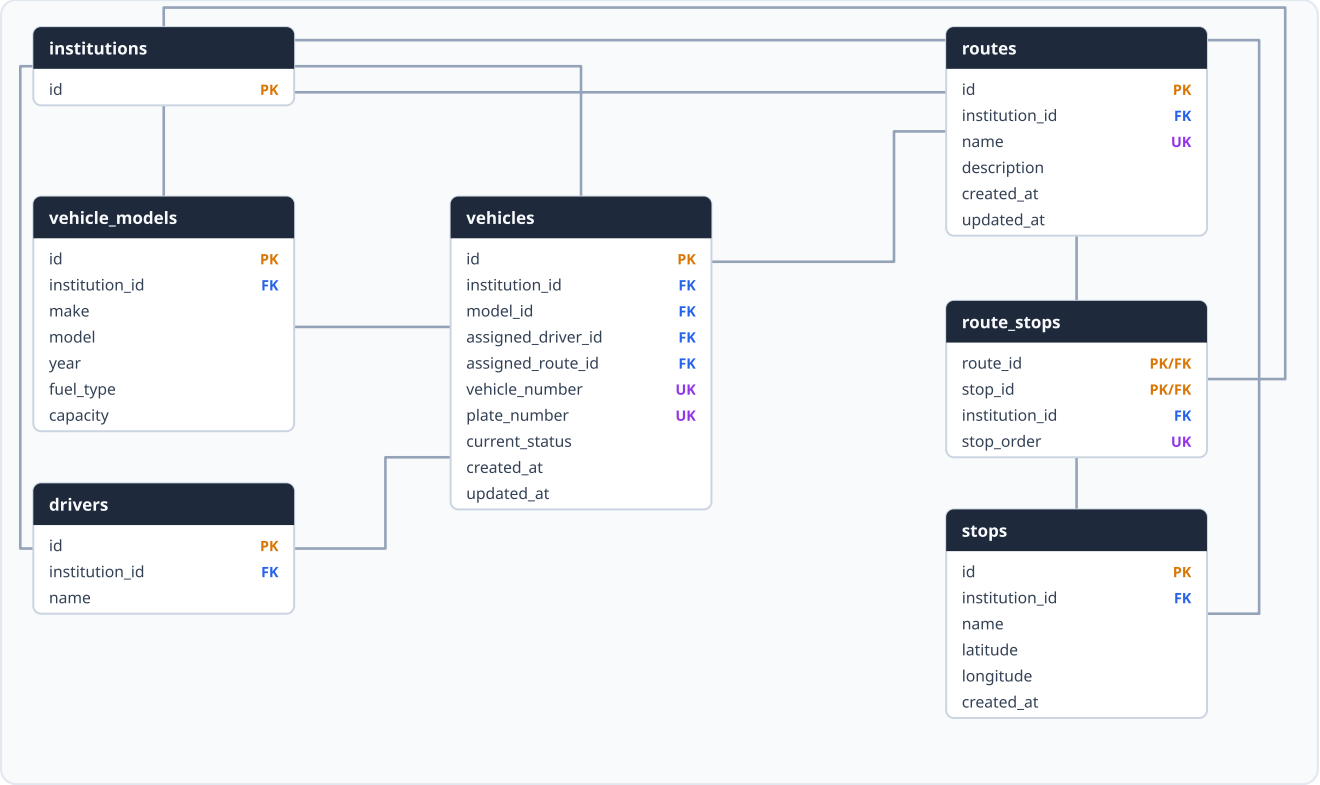
View 1: Tenancy, Users and Access

Covers the core tenant entity, subscriptions, employee identities (users and drivers), and the comprehensive Role-Based Access Control (RBAC) model.



View 2: Fleet and Assignments

Manages the physical assets, logical grouping into global or tenant-specific models, and spatial routing definitions.

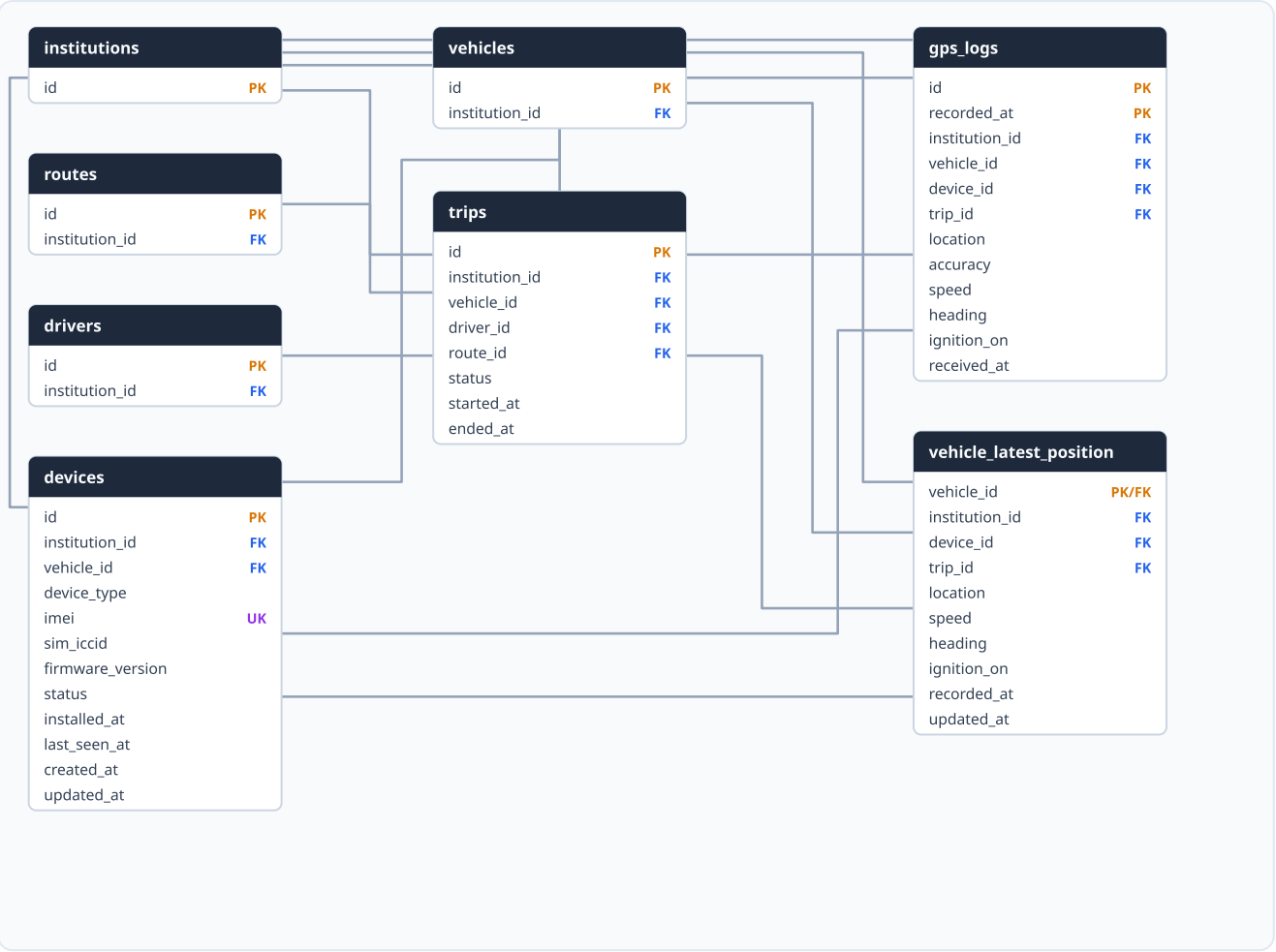


Assignment & Model Rules

The `assigned_driver_id` and `assigned_route_id` on `vehicles` are strictly optional, accommodating dynamic operational workflows like ride-hailing or on-demand delivery where a fixed route assignment does not apply. Because `vehicle_models` can operate as a global platform catalog (where `institution_id IS NULL`), enforcing safe references relies on checking functions or triggers rather than purely composite foreign keys.

View 3: Tracking and Trips

The real-time telemetry pipeline mapping physical devices to active vehicles, logging coordinate trails, and maintaining current map state.



GPS Log Volume Strategy

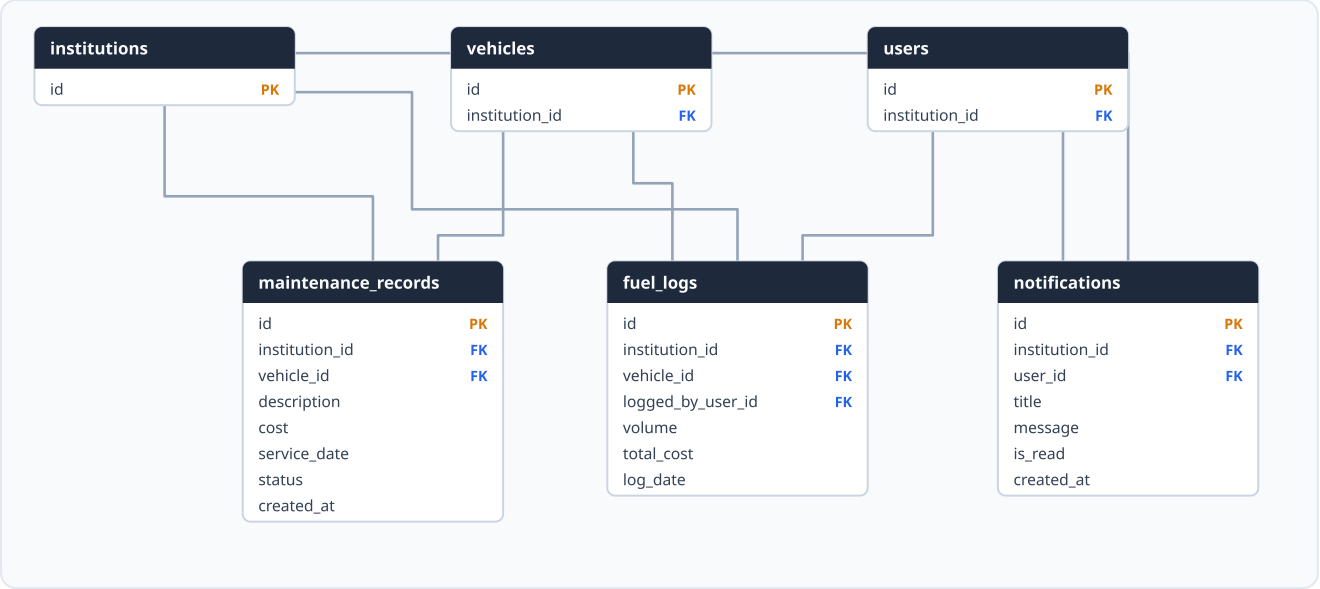
Because telemetry grows exponentially faster than core business entities, `gps_logs` enforces a dedicated volume management approach:

```
CREATE TABLE gps_logs (
    id             bigint GENERATED ALWAYS AS IDENTITY,
    recorded_at    timestamptz NOT NULL,
    institution_id uuid        NOT NULL,
    vehicle_id     uuid        NOT NULL,
    ...
    PRIMARY KEY (id, recorded_at)
) PARTITION BY RANGE (recorded_at);
```

The time-series partition ensures older chunks (e.g. `gps_logs_2026_10`) can be efficiently dropped or rolled up once retention thresholds have passed. The live state is strictly upserted into `vehicle_latest_position`, relieving query pressure on the raw log during live-map rendering operations.

View 4: Maintenance, Fuel and Notifications

Operational records logging recurring tasks, expenditures, and communication with system users.



Tenant Isolation Implementation Context

The schema relies strictly on PostgreSQL-level constraints to prevent data bleeding between tenants. By exposing a composite unique key `UNIQUE (institution_id, id)` on core entities, child tables inherently lock into the same institutional namespace when generating foreign keys.

```
ALTER TABLE vehicles ADD CONSTRAINT uq_vehicles_tenant UNIQUE (institution_id, id);

-- Complex routing ensures resources belong strictly to the active tenant execution context
ALTER TABLE trips ADD CONSTRAINT fk_trips_vehicle
    FOREIGN KEY (institution_id, vehicle_id) REFERENCES vehicles (institution_id, id);
```

Application logic should be coupled with connection-level Row Level Security (RLS) configured via `current_setting('app.current_institution_id')` to enforce these relationships seamlessly before queries execute.